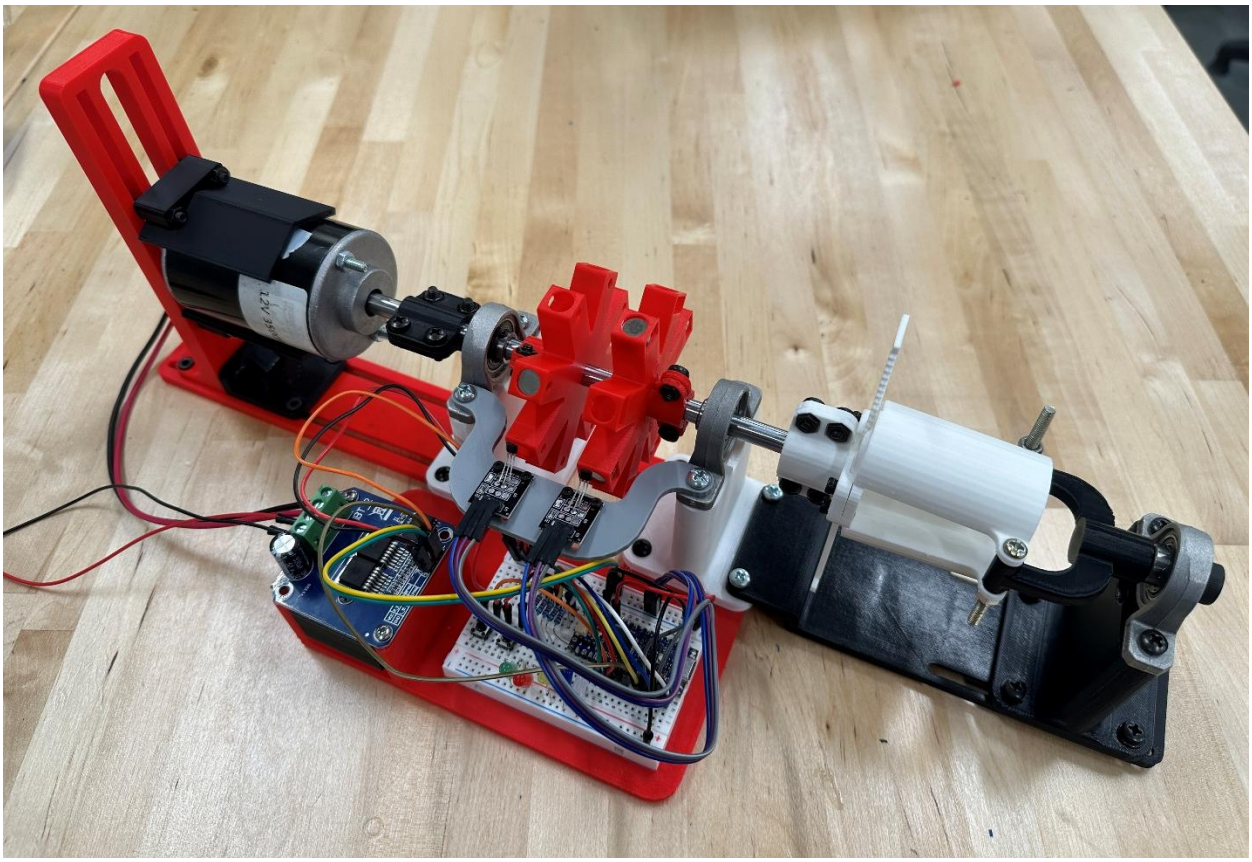
DN-G24N4D LABORATORY DYNAMOMETER



MANUAL AND OPERATING GUIDE

Group 10 Industries

4370 Polytechnic Cir, Lakeland, FL, 33805

Neil Hasenstaub, Ryan Mullins, Brennan Slatniske, Christian Sudduth

Table of Contents

Foreword

Thank you for your interest in the DN-G24N4D Laboratory Dynamometer! We hope this manual serves to aid your data collection from our instrument. As this dynamometer was developed in two months with a $100 budget (plus 3D printing), we understand its practical limitations and shortcomings. Nonetheless, it was fun to create as a project, and we hope you may learn something from using it as a real device.

-Neil Hasenstaub, Dynamometer Project Lead

1. Introduction

The DN-G24N4D Laboratory Dynamometer is an inertial dynamometer unit composed of a motor mount, Hall-effect tachometer, and a clamp for test samples. The electronics include a DC motor speed control, Arduino Nano R4 microcontroller, and button control embedded on the main board. The system operates by recording shaft speed over time to measure acceleration, then using dynamics principles to determine motor torque, moment of inertia, or shaft power.

1.1 Dynamometer Physics

An "inertial" type dynamometer is one which uses rotational acceleration of a known mass to determine torque and power of an engine or motor. The equation governing this is $T = I\alpha$, where T is torque, α is angular acceleration, and I is mass moment of inertia. Mass moment of inertia can be understood as the rotational equivalent of mass; it is a measure of an object's resistance to angular acceleration around a particular axis.

This system has this function as other inertial dynamometers offer, calculating torque from acceleration and a known moment of inertia. However, this unit is capable of the inverse; with a known torque/power of a motor, the moment of inertia of an object could be determined. This is achieved by adding an additional object to a universal clamp as a "test sample", which will increase the mass moment of inertia and thus the system's resistance to acceleration. This equation would thus be:

$$T = (I_{dyn} + I_{obj})\alpha \rightarrow I_{obj} = \frac{T}{\alpha} - I_{dyn} = I_{tot} - I_{dyn}$$

The unknown object's moment of inertia is thus the moment of inertia of the dynamometer components, a known value, subtracted from the total moment of inertia of the system.

Additionally, the power of a shaft can be calculated from the angular speed of the shaft and the torque. This relation is given by $P = T\omega$, where P is power and ω is angular velocity. The power of the motor could be estimated from the measured average angular velocity and the torque.

1.2 Intended Use

This dynamometer was designed to be multi-purpose. As previously mentioned, the system can measure both motor torque and moment of inertia of sample objects. The system can also function as a simple tachometer to measure the speed of a motor, as well as estimating power using the torque and motor speed. The mounts for both the motor and test object are intended to be as universal as possible, but due to system design, these parameters have limitations. These limitations will be listed in the system specifications.

2. Hardware Overview

The DN-G24N4D dynamometer comprises mechanical and electrical systems in a reasonably sized package. The instrument's architecture and parameters are broken down here for reference and understanding of its function.

2.1 Components & Layout

Figures 1 and 2 show labeled components of the dynamometer. Component numbers, names, and descriptions follow in Table 1.

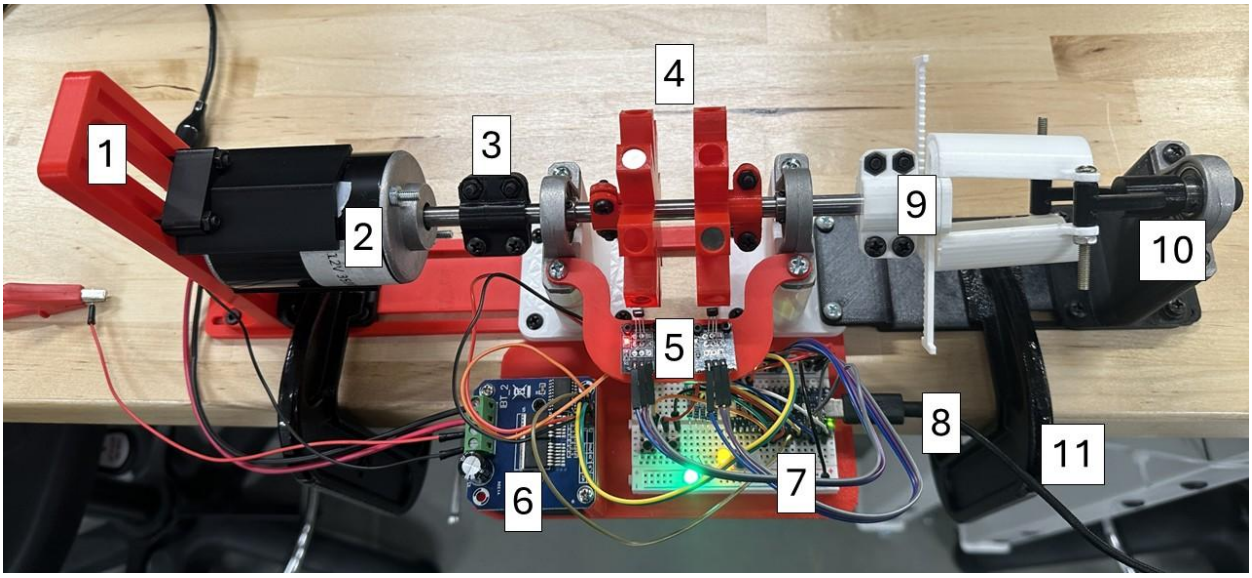


Figure 1: Full dynamometer with numbered components.

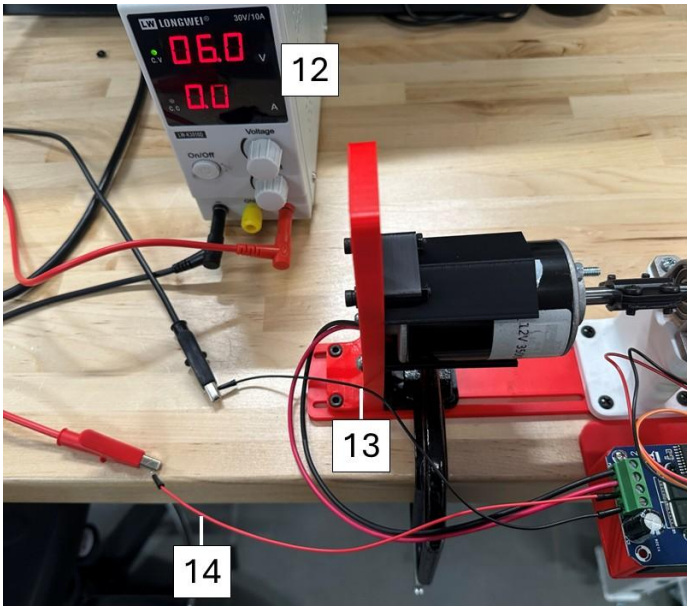


Figure 2: Dynamometer external components.

Table 1: Component names and descriptions. Numbers come from Figures 1 & 2.

#	Name	Description
1	Motor mount	Can adjust the height, length, and grip size to hold different motors.
2	Motor	Creates rotation of the shaft. Attached to shaft using coupler.
3	Shaft coupler	Connects motor to shaft. One will need to be supplied for the motor used, coupled from the motor shaft size to 8mm of the main shaft.
4	Magnet wheels	Rotational components which hold the magnets which activate the Hall-effect sensor.
5	Hall-effect sensors	Electric components which activate in the presence of a magnetic field of sufficient strength.
6	DC motor controller	Turns a PWM signal from the Arduino into a motor voltage.
7	Electronics breadboard	Contains the Arduino microcontroller, buttons, and LEDs.
8	USB-C cable	Connects the Arduino to PC.
9	Object clamp	Fits test sample objects to test their moment of inertia.
10	Clamp shaft	Keeps the object clamp centered on the shaft to reduce system vibration.
11	C-clamp	Clamps to fix the dynamometer in place. Two are required, one on each side.
12	Power supply	Supplies the motor with power through the motor controller.
13	Ground (Black) input wire	The ground connection from the power supply.
14	VCC (Red) input wire	The voltage source from the power supply.

2.2 System Control & Status

The dynamometer has two buttons and five LEDs, all located on the front of the breadboard, for system control and status monitoring. These are shown in Figure 3.

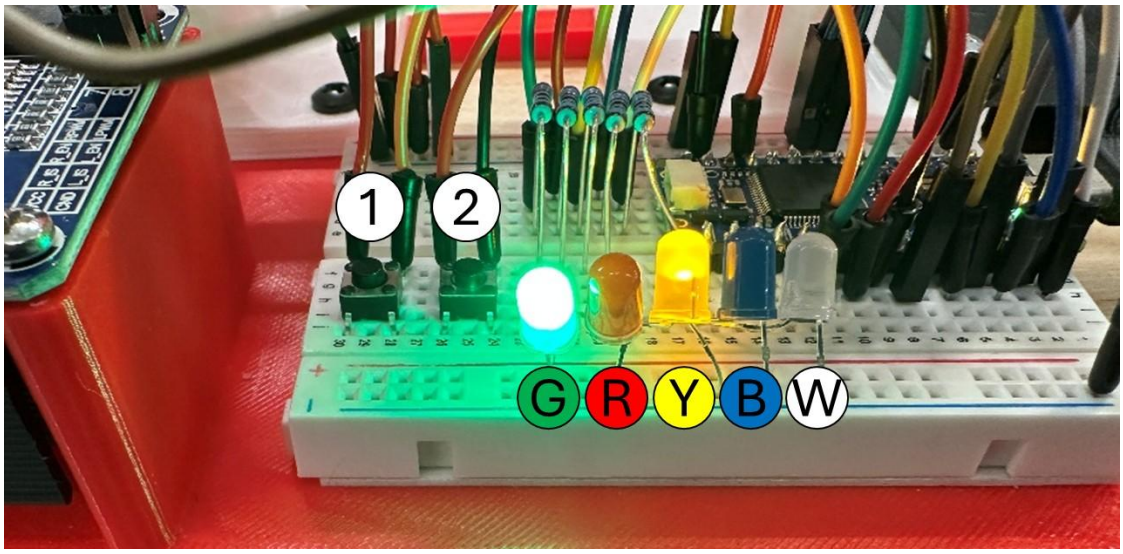


Figure 3: System buttons and LEDs, used for control and monitoring.

Each LED's signal is shown in Table 2.

Table 2: LED colors and functions.

Color	Indicates
Green (G)	Ready for use.
Red (R)	System is in use. Input is disabled.
Yellow (Y)	RPM mode is active. Motor will run for 10 seconds.
Blue (B)	Torque mode is active.* Motor will run for 5 seconds.
White (W)	Moment of Inertia mode is active.* Motor will run for 5 seconds.

Button 1: Mode Selection

Pressing the left button cycles the dynamometer modes. The order is RPM (default), Torque, then Moment of Inertia. The respective LED will light up, indicating which mode is currently active.

Button 2: Start Dynamometer

Pressing the right button starts the RPM data collection. The status LED will switch from green to red for the duration of the data collection period. Input from either button will be disabled when the motor is running.

*These modes are functionally identical, due to the same running time for RPM collection.

2.3 Specifications

2.3.1 Electronic Components

Microcontroller	Arduino Nano R4
Motor Controller	BTS-7960 H-Bridge DC Motor Driver
Tachometer Sensor	2x A3144 Hall-effect Sensor
Interface Method	Serial over Arduino USB-C
Serial Baud Rate	9600 baud

2.3.2 Power Ratings

USB-C Voltage	5V DC
USB-C Current	<100mA
BTS-7960 Voltage	6V-27V DC
BTS-7960 Current	<43A

2.3.3 Size Constraints

Max Motor Diameter	90 mm, from shaft center
Max Motor Length (total length inc. shaft)	150 mm
Max Width of Object Clamp	31.5 mm
Max Length of Test Object (mounted along rotation axis)	60 mm
Max Length of Test Object (mounted transversely)	120 mm total 60 mm, radius from shaft center

2.3.4 Range, Resolution & Uncertainty

RPM Range	0 – 25,000 RPM
RPM Resolution	$\frac{8}{200000} RPM^2$
RPM Uncertainty	½ * resolution
Torque Accuracy	±0.0021 Nm
Moment of Inertia Acc.	±5.76 kg m^2
Power Accuracy	±1.08 W

3. Software Overview

The DN-G24N4D dynamometer includes DynaCode, a complementary software package for collecting, managing, and processing data.

3.1 Introduction to DynaCode

DynaCode is a console-based Python application which links the dynamometer's Arduino to a PC. It is designed as an all-encompassing suite to receive and package data from the dynamometer, as well as offer several utilities for managing the system. Contained within the application is DynaLab, a graphical MATLAB Applet for calculating the relevant dynamics parameters from the collected RPM data. The entire application is self-contained; DynaLab is launched from within DynaCode, as will be explained later.

3.1.1 Software Requirements

The computer that is used is recommended to have Windows 11. Windows 10 is compatible, but terminal color formatting will not work properly.

MATLAB version R2025b is required. No other version is compatible with the DynaLab application. This is due to how MATLAB compatibility functions.

The Arduino IDE is not strictly required but is strongly recommended for quickly finding the serial port number.

3.1.2 Starting DynaCode

DynaCode is a stand-alone application; only the executable is required to run. It will run as a single file, without needing an installer or Python.

After running the application, Windows may give a popup about security risk in running an unsafe program. This is done because it is an application from an unknown publisher. Click "Run anyway" to start the application. The application cannot modify files in your computer; the extent of its ability is to read the contents of a folder and create CSV files.

The latest version of the application can be downloaded here.

3.1.3 Navigating the Console Interface

DynaCode operates as a command-line program, meaning the only input is through the keyboard into a terminal. Most input cases for DynaCode operation will require either a single letter or a line of text, depending on the situation. An example of each of these cases is shown in Figure 4.

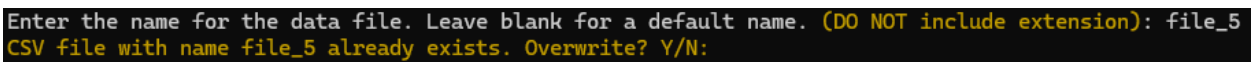


Figure 4:. Examples of full text line (top) and single letter (bottom) user input cases.

Single letter inputs require only the letter to be entered to select that option. Both upper and lower cases are accepted. Full text lines are needed when a name should be specified. Anything could be entered here, but there may be external restrictions. Any such restrictions will be mentioned when relevant.

You may encounter the instruction “Press enter to continue”. This is most often performed as a “check” if the user is ready to proceed. Due to how the application was developed, you will be able to enter text here, but the text will not be stored or otherwise used.

It is important to note that for reason of failsafe, entering nothing (by simply pressing “Enter”) or something other than the available options at an input point will be considered the “no” or “exit” option. This practice should be avoided, and only the exact input should be entered to prevent undesired behavior.

3.1.4 Text Color

While using DynaCode, you may encounter colored text. Table 3 shows the intended meaning of text coloring for reference.

Table 3: Text color interpretation.

Green	Success
Red	Error/Failure
Yellow	Warning
Blue	Informational
Purple	Heading/Label

3.2 Initial Setup

Upon launching DynaCode, you will be shown a window like that in Figure 5.

Figure 5: DynaCode window after opening.

Before using DynaCode with the dynamometer, two initialization steps are required: serial connection and folder selection.

3.2.1 Serial Connection

The first thing the program asks for is the serial port number of the Arduino connection to the PC. The easiest way to determine this is to open the Arduino IDE and select the connected board within the code editor. Under the board dropdown, the name of the connected board will be shown, along with its port number. This process is described in detail in Section 4.2.1.

The port is referred to as “COM#”, where # is the number. Only the number should be entered here; including COM or anything else is invalid and will prompt for the number again.

3.2.2 Folder Selection

Following a successful serial connection, a folder will need to be selected. This will be the location where the RPM data files will be stored. It is recommended to select a folder that is quick to navigate to, as the folder will need to be accessed later for data processing with DynaLab.

Press enter to open the file selection dialog. After selecting a folder, you will be asked to confirm the folder. Note you will need to re-focus the terminal window, as control is not passed back automatically. If you select “no” or fail to select a folder (by exiting the dialog), you will be prompted to select again. To prevent the dialog from appearing again instantly, you will have to press enter again to continue. The folder selection can be changed again later, if necessary.

After folder selection, you will enter the main menu.

3.3 Functions

The DynaCode main menu offers several functions for preparing RPM data, as well as other maintenance options. The main menu is shown below in Figure 6.

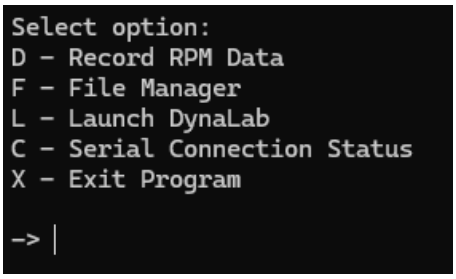


Figure 6: Main menu prompt.

To select an option, enter the letter associated with that option.

3.3.1 Record RPM Data

Entering “D” will begin the setup for collecting RPM data.

First, a name for the CSV data file must be specified. Most names are acceptable, limited only by the invalid Windows file names. These should be referenced externally if necessary. Entering a blank name here will give the file a default name.

Overwrite protection is present if a CSV file with the provided name already exists. If this happens, you will be prompted to confirm overwriting of the file. Selecting no here will ask for a new name.

Once the file is created, DynaCode will await data sent from the dynamometer. This process begins when the start button is pressed, which sends a start signal to the program. Data will stream into the terminal, with each entry being displayed on the screen. Due to high frequency of samples taken, the data lines will flash quickly.

PHOTOSENSITIVITY WARNING: FAST SCROLLING DATA MAY BE UNCOMFORTABLE FOR SOME. DISCRETION IS ADVISED.

Once data collection is complete, you will be prompted to run data collection again. Selecting yes here will repeat this process, starting from entering a file name. Selecting no will return to the main menu.

3.3.2 Folder Manager

Entering “F” will open the folder manager menu. Several options relating to the data storage folder are available. Options are selected the same way as the main menu.

- Entering “F” lists all files inside the folder in the terminal.
- Entering “C” allows you to change the folder. The process is the same as in the initial setup.
- Entering “O” opens the folder in File Explorer.

Entering “X” here returns to the main menu.

3.3.3 Launching DynaLab

Entering “L” will launch DynaLab, the data processing application. After a brief splash screen, the program will appear. Section 3.4 describes DynaLab’s full operation and capabilities.

After opening DynaLab, DynaCode will remain open and await an input of enter to return to the main menu. DynaCode can still be used with DynaLab open. DynaLab can also be opened multiple times, which is not recommended.

3.3.4 Serial Connection Status

Entering “C” will display the status of the serial connection, including the port and baud rate. The serial connection can also be temporarily disconnected and reconnected, for example if the Arduino cable needs to be re-plugged.

- Enter “D” to sever the serial connection.
- Enter “R” to re-establish the serial connection.

Entering “X” here will return to the main menu. Note that you will not be able to return to the main menu until a serial connection is established.

3.4 DynaLab Sub-App

DynaLab is a MATLAB GUI application used to process RPM data. The data is read from the CSV file to determine acceleration; it can then be used to calculate torque or moment of inertia given one or the other, as well as average RPM over the measurement time.

3.4.1 GUI Overview

The major features of DynaLab are shown in Figure 7 with labels. The numbers, names, and descriptions are provided in Table 4.

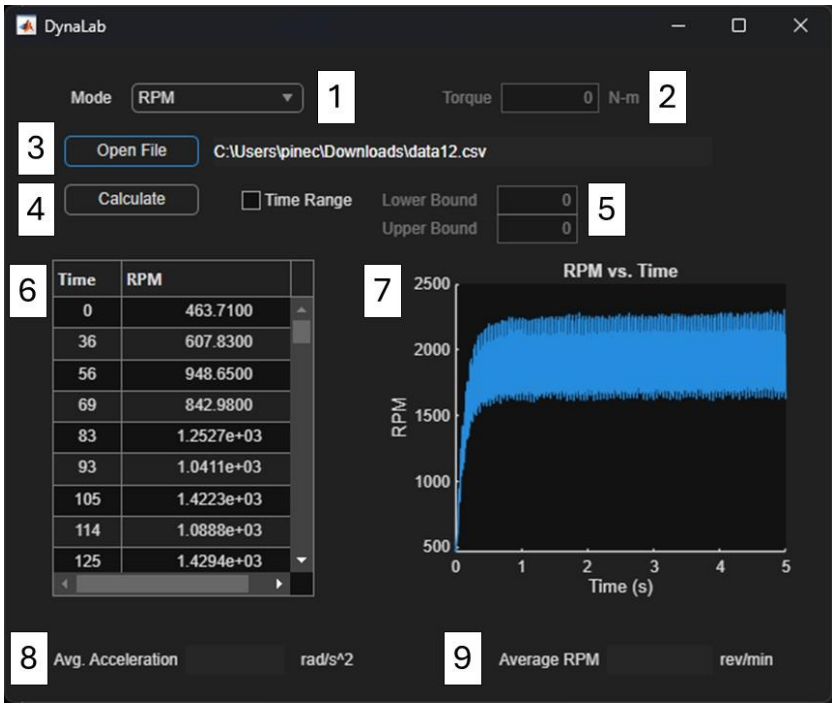


Figure 7: DynaLab GUI with loaded sample data.

Table 4: DynaLab features and descriptions. Numbers come from Figure 8.

#	Name	Description
1	Mode Dropdown	Selects the parameter to calculate based on the data.
2	Input Value	The value of torque or moment of inertia to calculate one or the other.
3	Open File Button	Opens a CSV file and reads the data into the table and graph.
4	Calculate Button	Calculates the average acceleration and parameter specified by the Mode Dropdown.
5	Upper/Lower Bounds	Specifies a time range for which the data is graphed and acceleration calculated. Enabled with the Time Range checkbox.
6	Data Table	Table with the CSV data.
7	RPM Graph	Graph containing RPM vs time from the data table.
8	Average Acceleration	Output for calculated average acceleration.
9	Calculated Parameter	Output for calculated RPM, torque, power, or moment of inertia.

3.4.2 Loading Data

Data is loaded by reading a CSV file. Press “Open File” to open the file selection dialog. Only CSV files created by DynaCode are permitted to be selected. Files with invalid formatting will report “Invalid file” and will not show any data.

If the file is valid, the time and RPM data will be shown in the table and plotted in the graph. The graph’s range can be modified with the Upper and Lower bounds, after checking the Time Range checkbox. The graph will be updated automatically. Note that this also affects how much of the data the acceleration and average RPM are calculated from.

3.4.3 Calculating Output

Output values are calculated directly from the input parameter and RPM data. The average acceleration is calculated in all modes, but output parameter is determined by the Mode Dropdown.

- RPM mode calculates average angular speed in RPM.
- Torque outputs a torque value based on an entered moment of inertia.
- Moment of inertia calculates the *total* moment of inertia of the system.

As previously mentioned, the calculated values will be based on the specified time range. If the range is not specified or set to (0,0), it will use the full range.

Press “Calculate” to output the values. The acceleration is then found in the bottom left, and the output parameter in the bottom right.

3.5 Exiting the Application

Once data collection is complete, return to the main menu. You can enter “X” to quit the application from here, where you will be prompted to confirm quitting with yes/no. If you select no, you will be returned to the main menu. If you select yes, you will be prompted to press enter a final time to successfully close the application.

Note that you could close the application at any time using the window options, but this is generally discouraged.

4. Operation

The following is the sequence of steps necessary to set up and run the dynamometer, collect RPM data with DynaCode, and process the data in DynaLab.

4.1 Physical Setup

The dynamometer must be mechanically configured for optimum use.

4.1.1 Materials List

The following materials should be prepared.

- DN-G24N4D Inertial Dynamometer
- 2x Large C-clamps
- USB-C to USB-A
- Power supply
- Alligator clips
- Motor (should be included in dynamometer)
- Test sample(s)
- PC with Arduino IDE and DynaCode
- Phillips and flathead screwdriver
- Hex M2.5 & M3.0 Allen keys

4.1.2 Securing the Machine

First, ensure that the dynamometer is fully assembled. Components should be as secured as tight as possible without destroying any threads. Ensure nothing is loose, and the dynamometer shaft spins easily with little resistance other than the motor.

The dynamometer should also be securely mounted to a table. This is easily done with two large C-clamps; one is under the motor and the other under the object clamp. The clamping setup, as well as the entire machine, is shown in Figure 8.

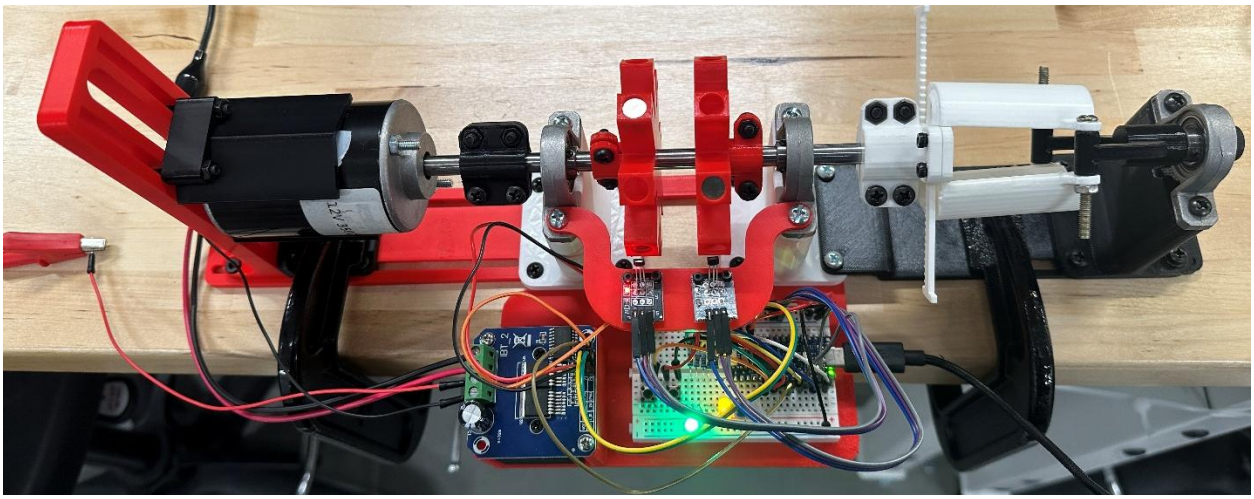


Figure 8: Dynamometer setup, without object in clamp.

4.1.3 Using the Object Clamp

The object clamp is designed to hold various test samples for testing their moment of inertia around the rotational axis. The test sample must be properly secured to provide an accurate measurement.

If no sample is to be secured, the nuts on the right end of the clamp should be screwed all the way down to close the clamp over the shaft bracket. Note that this will cause the brackets to bend; this is normal and does not damage the clamp. This is shown in Figure 9 (right).

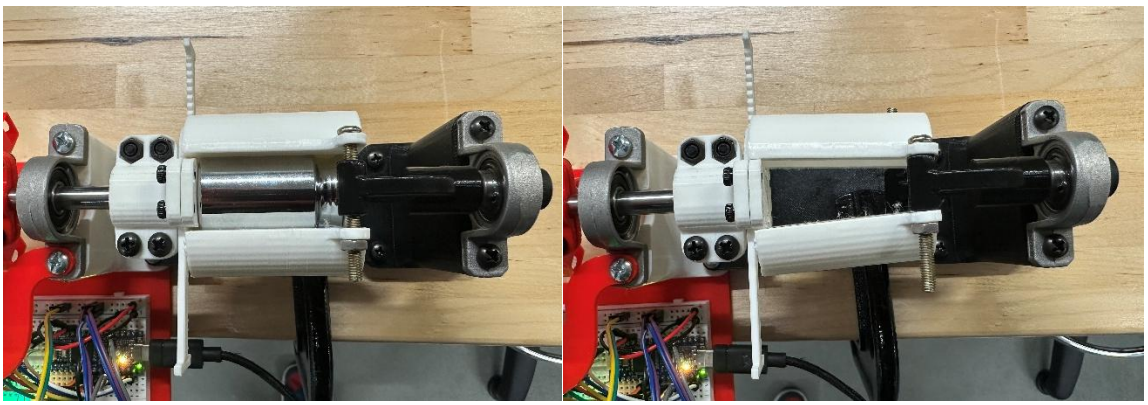


Figure 9: Object clamp. Configurations for loaded (left) and unloaded (right) are shown.

Round objects, such as that of Figure 9 (left), are easiest to secure due to the divots in the brackets, which align the object with the shaft axis. However, any object could be mounted, even transversely to the mount. Refer to Section 2.3 for size restrictions.

To mount an object to the clamp:

1. Unscrew the nuts at the end of the clamp to the end of their screws. This can be done by holding the nut and loosening the screw (Phillips).
2. Loosen the screws (M2.5) on the back of the clamp, on the shaft side. This reduces the friction of the clamp brackets.
3. Pull the brackets away from the shaft. Due to a geared design, the brackets will separate an equal distance.
4. Slot the object in from above, with the gap facing vertical like in Figure 9.
5. Press the brackets together over the object.
6. Tighten the screws (M2.5) on the back of the clamp, locking the brackets in place.
7. Screw the nuts over the brackets until they are tight.

If the object is not centered by the brackets automatically, try to make sure it is as evenly centered over the shaft as possible. If it is not perfectly centered, the machine will still work, however vibrations will be amplified due to dynamic imbalance, as well as reporting potentially higher moment of inertia.

4.1.4 Mounting a Motor

The configuration shown in this manual features a 12V 3500RPM DC motor. This motor is the standard unit for testing moment of inertia and calibrating its torque. However, this motor could be replaced with any other to test torque and power through use of the universal motor mount.

Mech Lab Note: Use the existing motor for the experiment, as its torque will be estimated through calibration.

If another motor is to be used, first ensure that you have a shaft coupler for the motor you wish to test. Couplers for select motor shaft sizes were created for testing; if the necessary size is not available, one could be easily created by modifying and 3D printing the template SolidWorks part.

The back of the motor mount is shown in Figure 10 for reference in the following procedure.

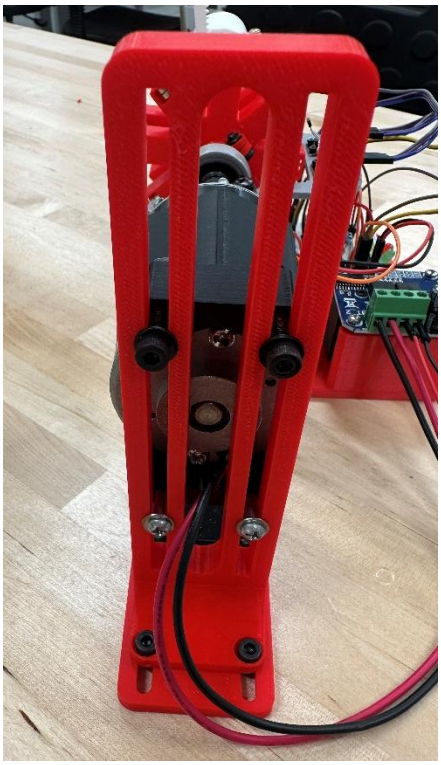


Figure 10: Back of motor mount.

The existing motor must then be decoupled and removed from the mount. The process is as follows.

1. Release the coupler by loosening the two screws (Phillips) on the motor side.

2. Loosen the four screws (Hex M3.0) on the bottom of the motor mount.
3. Slide the entire motor mount back. If the motor has a different shaft size, replace the coupler.
4. Loosen the top two screws (Hex M3.0) on the back, enabling the top bracket to slide up.
5. Remove the motor wires from the motor driver wire by unscrewing the blocks (flathead).
6. Remove the motor.
7. Install the new motor on the bracket. If it is not aligned with the shaft, loosen the bottom bracket (Phillips) and slide the motor up until it is aligned.
8. Secure the top bracket over the motor.
9. Slide the motor mount forward until the shaft is inside the coupler.
10. Clamp the coupler to the motor shaft.
11. Lock the motor mount by tightening the base screws. Tighten the brackets over the motor.
12. Connect the motor wires to the motor driver. If the motor driver is oriented with the green wire block on the left, the top most wire is the motor negative. The second from the top is motor positive. Refer to Figure 12 in Section 4.1.6 for connection configuration.

If everything is set up correctly, the motor should not be able to spin inside the clamp, and the dynamometer shaft should spin with the motor (unless the motor cannot be back-driven, in which case the shaft should be static).

4.1.5 Magnet Alignment

It is critical that the magnets on the tachometer are properly aligned, as this will affect the accuracy of the reported RPM values.

The DN-G24NR4D dynamometer has two rows of 4 magnets; for proper operation, the two wheels must be offset 45 degrees from each other. In this way, the system acts as one wheel with 8 magnets. Such a configuration causes the sensors alternate activation, which increases the theoretical maximum speed that can be read.

Each wheel was designed to hold 8 magnets. Since each wheel only holds 4 magnets in this setup, the wheels can be easily aligned by lining up the empty space of one wheel with a magnet horizontally. This is shown in Figure 12.

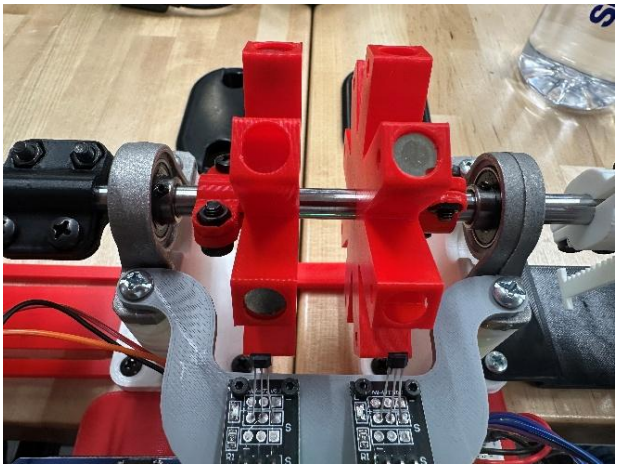


Figure 11: Ideal magnet wheel offset.

The wheels are secured to the shaft with a clamp-fit design. Even when tightened, the wheels should be able to slide around the shaft to precisely align them. If not, the wheel can be freed by loosening the clamp screws (Phillips) located to the sides of the wheels. Make sure the wheels are tight on the shaft when the system is to be used.

4.1.6 Power Setup

The motor driver has two wires coming out of it: one red and one black. These are the positive and negative power input leads, respectively. These should be connected to the positive and negative leads of the power supply.

IMPORTANT: Before turning on the power supply with the motor drive connected, ensure the power supply is set to 12V and 0A. High voltages and/or currents could severely damage the driver. Refer to Section 2.3 for power rating information.

Additionally, the motor's wires must be connected to the right blocks on the motor driver. Figure 12 shows the motor driver with its wire blocks labeled.

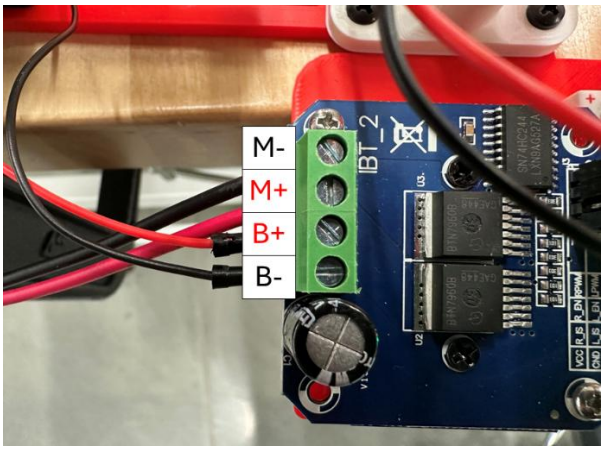


Figure 12: Motor drive wire block with pinout.

The positive (usually red) lead of the motor should be connected to the M+ block, and the negative (usually black) lead should be connected to the M- block.

4.2 Software Setup

Ensure the latest version of DynaCode is available on your PC. Connect the dynamometer's Arduino to the PC with a USB-C to USB-A cable.

4.2.1 Arduino Port

As described in Section 3.2.1, the Arduino port must be determined.

1. Open Arduino IDE.
2. Click the "Select Board" dropdown in the top left.
3. Note the port the Arduino is connected on; it should resemble "COM#".

If the board is not available, check Arduino connection. Refer to Section 5 for more information.

4.2.2 DynaCode Setup

To set up DynaCode for use:

1. Launch DynaCode from its executable. Refer to Section 3.1.2 for the download link.
2. Enter the port number acquired from the previous section.
3. Select an easily accessible folder using the dialog.
4. Re-focus the window and confirm folder selection or select a different folder.

The folder can be changed later if necessary; see Section 3.3.2. Additionally, the serial connection can be interrupted if necessary; see Section 3.3.4.

4.3 Using the Dynamometer

Before proceeding, make sure the correct motor and test object (if there is one) are mounted. Review Sections 4.1.3 and 4.1.4 for these procedures.

To collect a set of RPM data:

1. Select the correct mode with the left button. Refer to Section 2.2 for mode selection and LED status.
2. In DynaCode, from the main menu, select "Record RPM Data" by typing "D" into the console.
3. Provide a name for the file.
4. Verify the terminal reports "Awaiting Arduino Output". This is shown in Figure 13.

```
Select option:
D - Record RPM Data
F - Folder Manager
L - Launch DynaLab
C - Serial Connection Status
X - Exit Program

-> D

Arduino Serial to CSV

Enter the name for the data file. Leave blank for a default name. (DO NOT include extension): file_1
CSV file created: file_1.csv

Awaiting Arduino output...
```

Figure 13: DynaCode window entering data collection mode. Note "Awaiting Arduino Output" is the program waiting for the serial output to start.

5. Press the right button to start data collection.

Note: Keep all wires and loose objects away from the dynamometer.

6. If more than one data set is to be collected, enter “Y” at the prompt to continue, then repeat steps 3-5.
7. Once data collection is complete, enter “X” to return to the main menu.

4.4 Data Processing

Data processing is performed within DynaLab. The general process is as follows:

1. Launch DynaLab from the DynaCode main menu by entering “L”.
2. Once opened, click the “Open File” button. Select a data set from the previously specified location.
3. Verify data was correctly loaded into the table and graph. A sample of RPM data is shown in Figure 14.
4. If a specific time range is to be examined, enable the Time Range checkbox. Set the upper and lower bounds in their respective text boxes.
5. Select mode with the Mode dropdown in the top left. This changes which value is provided at the bottom right.
6. Input a value in the top right text box (where applicable)
7. Click the Calculate button to produce the results at the bottom. The average acceleration over the time range is always provided.

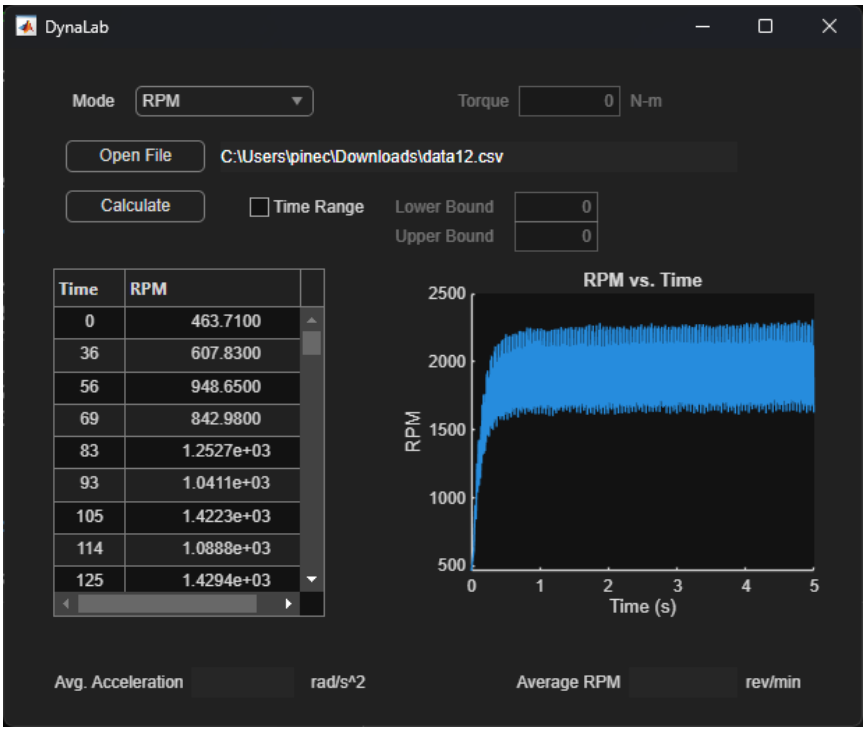


Figure 14: DynaLab with sample data.

It is worth noting that the calculated values are *very* sensitive to the time range over which they are calculated. For example, average acceleration calculated over the full range in Figure 14 will be much lower than if it were calculated over only the first second.

The different modes are described in the following four sections. The recommended range for each will be discussed.

4.4.1 RPM Mode

This mode simply calculates the average shaft speed over the specified time range. For the most accurate steady state speed, a range after acceleration (upward sloping region at the beginning) should be examined. For example, the range from t=3 to t=5 seconds in Figure 14 would be averaging the values after it has stopped accelerating.

Note this is the only mode that does *not* require an input at the top right.

4.4.2 Torque Mode

Torque mode takes in the calculated moment of inertia of the sample in the clamp and returns the torque of the motor.

Since this relation depends on the angular acceleration, the region of acceleration must be examined. The range should be set from the start of the data to the point where it approximately stops accelerating.

Setting the moment of inertia to zero assumes no additional mass inside the clamp.

4.4.3 Moment of Inertia Mode

Moment of inertia mode takes in the torque of the motor and returns the moment of inertia of the object in the clamp.

Like in Torque mode, the region of acceleration must be sampled. The range should be set from the start of the data to the point where it approximately stops accelerating.

The torque value should be non-zero, as this would assume the system did not have any power input. Having no object in the clamp and entering the estimated torque should theoretically return 0.

4.4.4 Power Mode

Power mode estimates the average power delivered by the motor. It takes in the moment of inertia of the object in the clamp calculates power based on the acceleration and the average motor speed over the sample period.

Like in the previous two modes, the region of acceleration must be sampled. The range should be set from the start of the data to the point where it approximately stops accelerating. The angular speed used in the power calculation is the average speed over the acceleration period, which will likely be much lower than the steady-state speed.

Like in Torque mode, setting moment of inertia to zero assumes no additional mass inside the clamp.

4.5 Sample Exercises

The following is a set of procedures which demonstrate the dynamometer's functions. These are for use with a XD-3420 12V DC motor.

Mech. Lab Note: This is the motor installed in the system.

4.5.1 RPM vs. Voltage

This procedure will explore the relationship between voltage input of the motor and the resulting speed.

1. Remove any test object from the clamp.
2. Set the dynamometer to RPM mode. (Yellow LED)
3. Set the power supply to 6V and 0A.
4. Run the dynamometer to get RPM data at 6V.
5. Repeat steps 2 and 3 for 7.5V and 9V.
6. Open each data set in DynaLab and record steady state RPM for each run.
7. (Optional) Plot RPM against voltage in an external program, such as Excel.

Voltage (V)	RPM
6	
7.5	
9	

4.5.2 Torque Calibration

This procedure will estimate the torque of the motor testing the system's behavior without a test sample.

1. Remove any test object from the clamp.
2. Set the dynamometer to Torque mode. (Blue LED)
3. Set the power supply to 6V and 1A.
4. Run the dynamometer three (3) times to get three (3) trials of RPM data.
5. In DynaLab, set the mode to Torque. Set moment of inertia to zero.
6. Set the time range to the acceleration period. A starting point could be 0 – 0.5 seconds, but the range should be tuned to capture all of the acceleration.
7. Calculate the estimated torque for each run and the average of the runs.

Trial	Torque
1	
2	
3	
Average	

4.5.3 Moment of Inertia of a Weight

This procedure will estimate the moment of inertia of a 200g balance calibration weight.

1. Insert the 200g balance calibration weight into the object clamp.
2. Set the dynamometer to Moment of Inertia mode. (White LED)
3. Set the power supply to 6V and 1A.
4. Run the dynamometer three (3) times to get three (3) trials of RPM data.
5. In DynaLab, set the mode to Moment of Inertia. Set the torque to 0.01 N-m.
6. Set the time range to the acceleration period.
7. Calculate the moment of inertia of the weight for each run and their average.
8. Calculate percentage error of the average moment of inertia compared the actual moment of inertia of 1.8496e-5.

Trial	Moment of Inertia
1	
2	
3	
Average	
% Error	

4.5.4 Power Comparison

This procedure will estimate the power output of the motor and compare the result with the theoretical output.

1. Remove any test object from the clamp.
2. Set the dynamometer to Torque mode. (Blue LED)
3. Set the power supply to 6V and 0A.
4. Run the dynamometer three (3) times to get three (3) trials of RPM data.
5. In DynaLab, set the mode to Power. Set moment of inertia to zero.
6. Set the time range to the acceleration period.
7. Calculate the estimated power for each run and the average of the runs.
8. The dynamometer has been observed to draw ~2A during acceleration. Compare 12W theoretical power the to the average power using percentage error.

Trial	Moment of Inertia
1	
2	
3	
Average	
% Error	

5. Troubleshooting

This section provides common troubleshooting information for the hardware and software.

Serial connection cannot be established.

Ensure the Arduino is connected to the PC with the USB-C to USB-A cable. Check port number using Arduino IDE.

Serial connection was interrupted before collecting data.

Attempt reconnection using terminal prompts. If this does not work, restart DynaCode. Check hardware connection by re-plugging USB cable.

Motor is not spinning when dynamometer is started.

Make sure the power supply is connected to the motor driver's power leads. The voltage input *must* exceed the minimum 6V for the motor driver.

Dynamometer causes excessive vibration.

Verify the voltage is not higher than necessary, as for some motors, this will give very high rotational speeds. Also, check that any object in the clamp is as closed to centered on the shaft as possible.

If the voltage is correct and object is secured, then the vibration is a side effect of imperfect dynamic balancing of the load. Ensure a sturdy connection to a table to prevent the system from moving in place.

Dynamometer does not stop spinning when started.

This behavior has been observed to happen very infrequently. If it does, turn off the power from the power supply. The Arduino should be restarted by either pressing the reset button on the Arduino (recommended) or re-plugging its USB connection.

Calculated moment of inertia is negative.

Verify input torque is of the correct magnitude. This results when the torque is very small, which would result in a small calculated total moment of inertia. Thus, when the constant moment of inertia of the system is subtracted, the result is negative.

Appendix

A.1 Arduino Code

The C++ code run on the Arduino is posted here for reference.

```
int GREEN_LED = 2;
int RED_LED = 4;
int YELLOW_LED = 8;
int BLUE_LED = 10;
int WHITE_LED = 12;

int MAGNET_1 = 11;
int MAGNET_2 = 9;
int MOTOR = 6;

int BUTTON_MODE = 3;
int BUTTON_START = 5;

int modeSwitchFlag = 0;
int startSwitchFlag = 0;
int firstRun = 0;

int RPM_RUNTIME = 10; //time in seconds for RPM to be sampled
int MOI_RUNTIME = 5;
int TORQUE_RUNTIME = 5;

void setup() {
  Serial.begin(9600);

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(BLUE_LED, OUTPUT);
  pinMode(WHITE_LED, OUTPUT);
  pinMode(MAGNET_1, INPUT);
  pinMode(MAGNET_2, INPUT);
  pinMode(MOTOR, OUTPUT);
  pinMode(BUTTON_MODE, INPUT_PULLUP);
  pinMode(BUTTON_START, INPUT_PULLUP);

  pinMode(LEDR, OUTPUT);
  pinMode(LEDG, OUTPUT);
  pinMode(LEDB, OUTPUT);

  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(YELLOW_LED, HIGH);
}
```

```
void loop() {
 delay(10);

 //tachometer mode, yellow LED
 if(modeSwitchFlag == 0) {
   digitalWrite(LEDR, LOW);
   digitalWrite(LEDG, LOW);
   digitalWrite(LEDB, HIGH);

   if(digitalRead(BUTTON_START) == 0) {
    do {
     delay(50);
    }
    while(digitalRead(BUTTON_START) == 0);

    double rps;
    unsigned long t0 = micros();
    unsigned long t0m = millis();
    startSwitchFlag = 1;

    digitalWrite(RED_LED, HIGH);
    digitalWrite(GREEN_LED, LOW);
    analogWrite(MOTOR, 255);

    Serial.println("begin");
    Serial.println("Time, RPM");
    firstRun = 1;
    while(micros() - t0 < RPM_RUNTIME*1000000) {
     Serial.print((millis() - t0m));
     Serial.print(",");
     rps = getRPS();
     if(firstRun == 1) {
      Serial.println(0);
      firstRun = 0;
     }
     else Serial.println(rps * 60);
    }
    Serial.println("end");

    for(int i = 255; i > 0; i-= 8) {
     analogWrite(MOTOR, i);
     delay(50);
    }
    analogWrite(MOTOR, 0);
    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);

    startSwitchFlag = 0;
```

```
  }
}

//torque mode, blue LED
else if(modeSwitchFlag == 1) {
  digitalWrite(LEDR, HIGH);
  digitalWrite(LEDG, HIGH);
  digitalWrite(LEDB, LOW);

  if(digitalRead(BUTTON_START) == 0) {
   do {
    delay(50);
   }
   while(digitalRead(BUTTON_START) == 0);

   double rps;
   unsigned long t0 = micros();
   unsigned long t0m = millis();
   startSwitchFlag = 1;

   digitalWrite(RED_LED, HIGH);
   digitalWrite(GREEN_LED, LOW);
   analogWrite(MOTOR, 255);

   Serial.println("begin");
   Serial.println("Time, RPM");
   firstRun = 1;
   while(micros() - t0 < TORQUE_RUNTIME*1000000) {
    Serial.print((millis() - t0m));
    Serial.print(",");
    rps = getRPS();
    if(firstRun == 1) {
     Serial.println(0);
     firstRun = 0;
    }
    else Serial.println(rps * 60);
   }
   Serial.println("end");

   for(int i = 255; i > 0; i-= 8) {
    analogWrite(MOTOR, i);
    delay(50);
   }
   analogWrite(MOTOR, 0);
   digitalWrite(RED_LED, LOW);
   digitalWrite(GREEN_LED, HIGH);

   startSwitchFlag = 0;
```

```
    }
  }

  //moment of inertia, white LED
  else if(modeSwitchFlag == 2) {
    digitalWrite(LEDR, LOW);
    digitalWrite(LEDG, LOW);
    digitalWrite(LEDB, LOW);

    if(digitalRead(BUTTON_START) == 0) {
     do {
      delay(50);
     }
     while(digitalRead(BUTTON_START) == 0);

     double rps;
     unsigned long t0 = micros();
     unsigned long t0m = millis();
     startSwitchFlag = 1;

     digitalWrite(RED_LED, HIGH);
     digitalWrite(GREEN_LED, LOW);
     analogWrite(MOTOR, 255);

     Serial.println("begin");
     Serial.println("Time, RPM");
     firstRun = 1;
     while(micros() - t0 < MOI_RUNTIME*1000000) {
      Serial.print((millis() - t0m));
      Serial.print(",");
      rps = getRPS();
      if(firstRun == 1) {
       Serial.println(0);
       firstRun = 0;
      }
      else Serial.println(rps * 60);
     }
     Serial.println("end");

     for(int i = 255; i > 0; i-= 8) {
      analogWrite(MOTOR, i);
      delay(50);
     }
     analogWrite(MOTOR, 0);
     digitalWrite(RED_LED, LOW);
     digitalWrite(GREEN_LED, HIGH);

     startSwitchFlag = 0;
```

```
    }
  }

  //MODE BUTTON PRESS
  if(digitalRead(BUTTON_MODE) == 0 && !startSwitchFlag) {
   do {
    delay(50);
   }
   while(digitalRead(BUTTON_MODE) == 0);

   switch(modeSwitchFlag) {
    case 0:
     modeSwitchFlag = 1;
     digitalWrite(YELLOW_LED, LOW);
     digitalWrite(BLUE_LED, HIGH);
     digitalWrite(WHITE_LED, LOW);
     break;
    case 1:
     modeSwitchFlag = 2;
     digitalWrite(YELLOW_LED, LOW);
     digitalWrite(BLUE_LED, LOW);
     digitalWrite(WHITE_LED, HIGH);
     break;
    case 2:
     modeSwitchFlag = 0;
     digitalWrite(YELLOW_LED, HIGH);
     digitalWrite(BLUE_LED, LOW);
     digitalWrite(WHITE_LED, LOW);
     break;
   }
  }

double getRPSmanual() { // revolutions per second, samples amount of revolutions per one second manually, much less efficient, unused in main function
  double detect1 = 0; // the number of magnets  that have been detected
  double detect2 = 0;
  bool detectFlag1 = true; // filters out the current detected magnet, it will not be counted
  bool detectFlag2 = true;
  unsigned long relativeTime = micros(); // reference time is established
  while(micros()-relativeTime < 1000000) { // sample period of one second
   if ((digitalRead(MAGNET_2) == 0) && !detectFlag1) { // if the first sensor is ON and has previously been off
     detect1++; // another magnet detected
     detectFlag1 = true;
   }
   if ((digitalRead(MAGNET_1) == 0) && !detectFlag2) { // if the second sensor is ON and has previously been off
     detect2++; // another magnet detected
```

```
    detectFlag2 = true;
   }
   if (digitalRead(MAGNET_2) == 1 && detectFlag1) { // if the first sensor if OFF and has previously
been on
    detectFlag1 = false; // ready to read another magnet
   }
   if (digitalRead(MAGNET_1) == 1 && detectFlag2) { // if the second sensor if OFF and has previously
been on
    detectFlag2 = false; // ready to read another magnet
   }
 }
 return(((detect1+detect2)/8.0)/((1.0))); // There are eight sets of magnets rotating, so one
revolution is eight "detects", and the RPS would be the revolutions divided by the elapsed time
}

double getRPS() { // samples the small time it takes to measure out 1/8 of a revolution, the double 8
magnet wheels act as a single 8 magnet wheel, assuming each magnet sweeps an equal amount of
arc length
 bool skipNext = false;
 do {
  continue;
 } while ((digitalRead(MAGNET_2) == 1 && digitalRead(MAGNET_1) == 1) || (digitalRead(MAGNET_2)
== 0 && digitalRead(MAGNET_1) == 0));
 unsigned long relativeTime = micros();
 if (digitalRead(MAGNET_2)==0 && digitalRead(MAGNET_1) == 1) { // whichever sensor reads on
first while the other is not
  while (digitalRead(MAGNET_2) == 0 && digitalRead(MAGNET_1) == 1) { // while the data stream is
reading ON and the other one is off
   continue;
  }
  skipNext = true; // prevents from counting the time of both wheels to activate and deactivate
 }
 if (digitalRead(MAGNET_2) == 1 && digitalRead(MAGNET_1) == 0 && !skipNext) {
  while (digitalRead(MAGNET_1) == 0 && digitalRead(MAGNET_2) == 1) { // while the data stream is
reading ON and the other one is off
   continue;
  }
 }
 if((micros() - relativeTime) == 0) {
  return 0;
 }
 else return 1/(8*(micros() - relativeTime)/1000000.0); // 1/8 of a revolution over the small time in
seconds returns the current speed in revolutions per second
}
```